

IP Addressing

1. IPv4 Address Basics

- **32 bits long** → address space = $2^{32} = 4,294,967,296$ addresses.
- Written in **dotted-decimal notation**: four 8-bit octets (0–255 each), separated by dots — e.g., **192.168.1.10**.
- **Validity rules** for a dotted-decimal address:
 - No leading zeros in an octet (**045** is invalid).
 - Exactly four numbers, each ≤ 255 .
 - No mixing binary and decimal notation in the same address.

Binary: 11000000.10101000.00000001.00001010
Decimal: 192 . 168 . 1 . 10

2. Classful Addressing

The original scheme divides the address space into five classes, identified by the leading bits:

Class	Leading bits	Range (first octet)	Default mask	Network/Host split
A	0	0 – 127	255.0.0.0 (/8)	1 octet network, 3 octets host
B	10	128 – 191	255.255.0.0 (/16)	2 octets network, 2 octets host
C	110	192 – 223	255.255.255.0 (/24)	3 octets network, 1 octet host
D	1110	224 – 239	—	Multicast (no network/host concept)
E	1111	240 – 255	—	Reserved (experimental)

Worked example — classifying addresses:

- **00000001 00001011...** → first bit 0 → **Class A**
- **11000001 10000011...** → first 2 bits **11**, third bit **0** → **Class C**
- **14.23.120.8** → first byte 14 (0–127 range) → **Class A**
- **252.5.15.111** → first byte 252 (240–255 range) → **Class E**

The first address in any block is the Network Address; the last is the Broadcast Address

— neither is assignable to an actual host.

Private (non-routable) address ranges

Reserved for use *inside* private networks only — routers on the public Internet won't forward packets to/from these:

Class	Private range
A	10.0.0.0 – 10.255.255.255
B	172.16.0.0 – 172.31.255.255
C	192.168.0.0 – 192.168.255.255

Loopback address

127.0.0.1 (the whole **127.0.0.0/8** block is reserved) — used to test a device's own network stack, without actually touching the network hardware.

3. Subnetting

Subnetting carves a single classful network into multiple smaller logical networks, by "borrowing" bits from the host portion to extend the network portion.

Why subnet at all? Without it, an organization granted (say) one Class B network is stuck with one enormous flat network of 65,534 hosts — wasteful, and a single broadcast domain that large is a performance and administration nightmare. Subnetting lets that same block be split into right-sized pieces.

Core formulas, given a prefix length **/n (or equivalently, a subnet mask):**

Number of host bits = $32 - n$
Number of usable hosts per subnet = $2^{(32-n)} - 2$ (–2 for network address and broadcast address)
Block size (subnet increment) = $2^{(32-n)}$
First address in a block = set all host bits to 0
Last address in a block = set all host bits to 1 (this is the broadcast address for that subnet)

Worked example 1: **255.255.255.128 (/25) applied to 192.168.10.0**

$128 = 10000000 \rightarrow 1$ bit borrowed for subnetting $\rightarrow 2^1 = 2$ subnets
 7 host bits remain $\rightarrow 2^7 - 2 = 126$ usable hosts per subnet

Subnet	Network address	Usable hosts	Broadcast address
1	192.168.10.0	.1 – .126	192.168.10.127
2	192.168.10.128	.129 – .254	192.168.10.255

Worked example 2: 255.255.255.224 (/27) applied to 192.168.10.0

224 = 11100000 → 3 bits borrowed → $2^3 = 8$ subnets
 5 host bits remain → $2^5 - 2 = 30$ usable hosts per subnet
 Block size = 256 - 224 = 32 → subnets start at 0, 32, 64, 96, 128, 160, 192, 224

Subnet	First host	Last host	Broadcast
0	.1	.30	.31
32	.33	.62	.63
...	...	...	...
224	.225	.254	.255

Worked example 3: given an address inside a block, find the block

A /28 block is granted; one address in it is 205.16.37.39 . Find the first and last address.

32 - 28 = 4 host bits
 205.16.37.39 in binary: 11001101 00010000 00100101 00100111
 Set rightmost 4 bits to 0 → 11001101 00010000 00100101 00100000 = 205.16.37.32 (first address)
 Set rightmost 4 bits to 1 → 11001101 00010000 00100101 00101111 = 205.16.37.47 (last address)

Block representation: 205.16.37.32/28 .

4. CIDR & VLSM — allocating addresses to differently-sized groups

CIDR (Classless Inter-Domain Routing) drops the rigid class boundaries entirely — any prefix length can be used, letting address blocks be sized to actual need rather than forced into A/B/C boundaries. **VLSM (Variable Length Subnet Masking)** is the technique of subnetting a single block into pieces of *different* sizes for different groups, rather than every subnet being the same size.

Worked example (full ISP allocation problem): An ISP is granted 190.100.0.0/16 (65,536 addresses) and must distribute it to three customer groups:

- Group 1: 64 customers, each needing 256 addresses
- Group 2: 128 customers, each needing 128 addresses
- Group 3: 128 customers, each needing 64 addresses

Step 1 — find the prefix each group's customers need:

Group 1: 256 addresses = $2^8 \rightarrow 8$ host bits $\rightarrow$ prefix = $32 - 8 = /24$
Group 2: 128 addresses = $2^7 \rightarrow 7$ host bits $\rightarrow$ prefix = $32 - 7 = /25$
Group 3: 64 addresses = $2^6 \rightarrow 6$ host bits $\rightarrow$ prefix = $32 - 6 = /26$

Step 2 — total addresses consumed by each group:

Group 1: 64 customers $\times$ 256 addresses = 16,384
Group 2: 128 customers $\times$ 128 addresses = 16,384
Group 3: 128 customers $\times$ 64 addresses = 8,192
Total = 40,960

Step 3 — addresses remaining:

Granted to ISP: 65,536
Allocated: 40,960
Remaining: $65,536 - 40,960 = 24,576$

Why VLSM matters in practice: without it, every subnet would be forced to the *same* size — meaning the ISP would have to give every customer the largest allocation any of them needs (256 addresses each), wasting enormous amounts of address space on customers who only actually needed 64 or 128. VLSM is precisely what makes efficient, need-based address allocation possible.

5. ARP (Address Resolution Protocol)

The problem ARP solves: IP addresses (logical, Network layer) get you to the right *network*, but actually delivering a frame on a local link requires the destination's **MAC address** (physical, Data Link layer). ARP is how a device discovers "which MAC address corresponds to this IP address, on my local network?"

Standard ARP Request / Reply

Ethernet header: SRC MAC = (sender's real MAC) DST MAC = FF:FF:FF:FF:FF:FF (broadcast)
ARP header: SRC IP = (sender's IP) DST IP = (target IP, MAC unknown)

- The **ARP request** is broadcast to everyone on the local network ("who has this IP? tell me your MAC").
- Only the device that actually owns that IP responds with a **unicast ARP reply**, filling in its own MAC address.

PC1 wants PC4's MAC:
PC1 $\rightarrow$ ARP Request (broadcast) $\rightarrow$ [everyone on the LAN]
PC4 $\rightarrow$ ARP Reply (unicast, direct to PC1) $\rightarrow$ PC1

Other ARP variants

- **Gratuitous ARP** — a device sends an ARP announcement **about itself**, unprompted, typically right after it boots/gets an IP. Used to detect if another device on the network is already using the same IP (a conflict), and to let other devices proactively update their own ARP caches.
- **Proxy ARP** — a router replies to an ARP request **on behalf of** another device (often on a different subnet), letting two devices that wouldn't normally see each other's ARP broadcasts still communicate as if they were on the same local network.
- **Reverse ARP (RARP)** — the inverse problem: a device knows its own **MAC** but not its **IP**, and asks the network to supply it. Historically used by diskless workstations at boot; effectively superseded by DHCP (see the next topic).
- **Inverse ARP (InARP)** — finds an IP address given *known virtual circuit information*, rather than a MAC — used in Frame Relay/ATM contexts.

Ping — ARP + ICMP working together

ping first needs a MAC address to actually deliver anything on the local segment, so the full sequence for a first-time ping is:

1. ARP Request → "who has target IP? tell me your MAC"
2. ARP Reply → target responds with its MAC
3. ICMP Echo Request → "are you alive?" (now that we know the MAC)
4. ICMP Echo Reply → "yes, I'm alive"

6. ICMP (Internet Control Message Protocol)

One-line definition: ICMP is used by network devices to report errors and exchange operational/diagnostic information — most famously, the **ping** (echo request/reply) mechanism, but also error reporting like "destination unreachable" or "TTL exceeded."

- Operates at the Network layer, alongside IP, but is used for *control/diagnostic* messages rather than carrying application data.
- A router that must drop a packet (e.g., because its **TTL** reached 0 — see the IPv4 Header topic) typically sends an ICMP "Time Exceeded" message back to the original sender, which is exactly the mechanism tools like **traceroute** exploit to map a packet's path hop by hop.

Interview Questions With Answers

Q1. Why was classful addressing eventually replaced by CIDR?

Answer: Classful addressing only offered three fixed network sizes for general use (Class A: ~16 million hosts, Class B: ~65,000 hosts, Class C: 254 hosts) — an organization needing, say, 500 addresses would either be under-served by a Class C or forced to waste most of a Class B's 65,000 addresses. CIDR removes the rigid class boundaries entirely, allowing any prefix length, so address blocks can be sized to actual need — this dramatically reduced the wasteful over-allocation that was rapidly depleting the IPv4 address space under the classful system.

Q2. Given a subnet mask, how do you calculate the number of usable hosts per subnet, and why do we subtract 2?

Answer: Usable hosts = $2^{(\text{host bits})} - 2$, where host bits = $32 - \text{prefix length}$. We subtract 2 because the very first address in any block is reserved as the network address (identifying the subnet itself, not any specific host) and the very last address is reserved as the broadcast address (used to reach every host in that subnet at once) — neither can be assigned to an individual device.

Q3. In the worked ISP example, why does Group 1 (needing only 256 addresses per customer) get a /24 while Group 3 (needing 64) gets a /26 — shouldn't more addresses mean a bigger prefix number?

Answer: It's the opposite relationship: prefix length counts *network* bits, and a *larger* prefix number means *fewer* host bits are left over, meaning a *smaller* block. Group 1 needs more addresses per customer ($256 = 2^8$), which requires more host bits (8), which means a *smaller* prefix number ($32 - 8 = 24$). Group 3 needs fewer addresses ($64 = 2^6$), requiring fewer host bits (6), giving a *larger* prefix number ($32 - 6 = 26$) and hence a smaller block per customer. More addresses needed always corresponds to a numerically smaller (less specific) prefix.

Q4. What is VLSM, and why couldn't the ISP allocation example be solved without it?

Answer: VLSM (Variable Length Subnet Masking) allows subnets carved from the same parent block to have *different* sizes, rather than forcing every subnet to be identical. The ISP example has three customer groups needing very different amounts of address space per customer (256, 128, and 64) — without VLSM, every customer across all three groups would have to be given the same, largest allocation (256 addresses each, to satisfy Group 1's need), which would have consumed the entire /16 block (65,536 addresses) just on Group 1's 64 customers alone, let alone the other 256 customers in Groups 2 and 3 — VLSM is precisely what makes it possible to right-size each group's allocation independently.

Q5. Why does ARP broadcast its request instead of sending it directly to the target?

Answer: The entire point of an ARP request is that the sender doesn't yet know the target's MAC address — and without a MAC address, there's no way to address an Ethernet frame directly to that specific device. Broadcasting the request (to the special all-FFs MAC address) ensures every device on the local network segment receives and processes it, and only the device that actually owns the queried IP address needs to respond, with a normal unicast reply once it has the requester's MAC (learned from the request frame itself).

Q6. What problem does gratuitous ARP solve, and when is it typically sent?

Answer: Gratuitous ARP solves IP address conflict detection and proactive cache updates — a device sends an unsolicited ARP announcement about its own IP-to-MAC mapping (typically right after booting or acquiring a new IP address), which serves two purposes: if another device on the network already claims that same IP, it will notice and can flag the conflict, and other devices on the network can proactively update their ARP caches with the new mapping without needing to separately query for it later.

Q7. Why does a first-time ping between two hosts on the same LAN generate ARP traffic in addition to ICMP traffic?

Answer: ICMP Echo Request/Reply (what ping actually uses) is a Network-layer exchange addressed by IP, but actually delivering that ICMP packet on the local Ethernet segment requires wrapping it in a frame addressed to a specific destination MAC address — which the sender doesn't yet have on a first-time ping. So the sender must first perform ARP resolution (broadcast request, unicast reply) to learn the target's MAC address before it can construct and send the actual ICMP Echo Request frame; subsequent pings to the same target typically skip the ARP step since the mapping is now cached.

Q8. Scenario: An ISP has a /20 block and needs to allocate address space to four departments needing 500, 250, 100, and 50 hosts respectively. Walk through how you'd size each department's subnet.

Answer: For each department, find the smallest power of 2 that's $\geq$ (needed hosts + 2, to account for network/broadcast addresses), then derive the prefix from the corresponding host-bit count: 500 hosts needs $2^9 - 2 = 510 \geq 500 \rightarrow 9$ host bits $\rightarrow /23$ ($32 - 9 = 23$); 250 hosts needs $2^8 - 2 = 254 \geq 250 \rightarrow 8$ host bits $\rightarrow /24$; 100 hosts needs $2^7 - 2 = 126 \geq 100 \rightarrow 7$ host bits $\rightarrow /25$; 50 hosts needs $2^6 - 2 = 62 \geq 50 \rightarrow 6$ host bits $\rightarrow /26$. You'd then assign these subnets within the /20 block, conventionally allocating the largest blocks first (VLSM's typical allocation discipline) to keep the address space cleanly aligned and avoid fragmentation — e.g., the /23 block first, then the /24, then /25, then /26 — checking that their cumulative size doesn't exceed the /20's total 4096 addresses (which it doesn't here: $512 + 256 + 128 + 64 = 960$, well within budget).